



Kaggle Competition Report

AI Assignment #03

Name:

Muhammad Abu Bakar

NU ID:

24K-0826

April 27th, 2026

—

Syllabus:

Artificial Intelligence Theory

Syllabus Code: AI-2002

—

Instructor:

Sir Usama Antuley

1. Competition Overview

- Competition Name:

Kaggle Playground Series – Predicting Irrigation Needs

- Problem Type:

Multiclass Classification

- Evaluation Metric:

Balanced Accuracy

- Brief Understanding of The Dataset:

The dataset contains 6.3 lakh rows of agricultural data like soil type, rainfall, and crops to predict if a field needs Low, Medium, or High irrigation.

The biggest issue with this dataset is a severe class imbalance:

Low is about 60% of the data, Medium is 37%, and High is only 3%.

2. Data Preprocessing

- Missing Values Handling:

The dataset was clean, EDA didn't have to be performed

- Encoding Techniques:

An OrdinalEncoder was used for the text categories like Soil_Type and Region. I chose this instead of OneHotEncoder because creating new columns for every category on a 630k row dataset would have taken up too much memory that my laptop probably doesn't have enough RAM to offer

- Feature Scaling:

The numerical columns were passed through directly since my best model Decision Trees doesn't strictly require scaling like distance based models do.

- Train Validation Split:

Instead of a simple split, Cross Validation was used to test the models on different chunks of data to see how good they actually were.

3. Models Attempted

- Decision Tree:

A model that makes decisions using a tree-like flowchart.

- Naive Bayes:

A basic model based on probability.

- K-Means as Classifier:

Wrote a custom code to group data into 3 clusters and assign the most common label to each group.

- Logistic / Linear Regression:

Used a Ridge Classifier to try and separate classes using straight lines.

- Any Advanced Models:

None

4. Cross Validation Strategy

- K-Fold Details:

I used 5-Fold Stratified Cross Validation. This splits the data into 5 equal parts. The stratified part is super important because it makes sure that the rare High irrigation class gets split evenly across all 5 parts so the model can actually see it and learn it.

- LOOCV Observations:

Did not opt for it, because doing that meant testing the model 6.3 lakh separate times. That would take hours on my computer, so 5 fold was a much smarter choice.

- Best Validation Accuracy:

The best accuracy during testing was about 96%, **95.98% to be precise**, with my tuned decision Tree.

5. Failed Attempts and Insights

a. Failed Attempt 1

- Model Used:

K-Means

- Accuracy Obtained:

33.33%

- Confusion Matrix Screenshot:



```
Fold 1/5 – balanced_accuracy = 0.3333
Fold 2/5 – balanced_accuracy = 0.3333
Fold 3/5 – balanced_accuracy = 0.3333
Fold 4/5 – balanced_accuracy = 0.3333
Fold 5/5 – balanced_accuracy = 0.3333
— K-Means Wrapper CV Summary —
Mean Balanced Accuracy : 0.3333
Total time              : 43.2s
```

```
Confusion Matrix (last fold)
Classes: ['High', 'Low', 'Medium']
[[ 0 4202  0]
 [ 0 73983 0]
 [ 0 47815 0]]
```

- What Went Wrong?:

K-Means is meant for grouping unlabelled data, not predicting specific labels. If we look at the matrix above (from the output I got when I ran the python file), it actually just guessed Low, the middle column, for every single row in the test! It completely failed to learn anything.

b. Failed Attempt 2

- Model Used:

Ridge Classifier (Linear Regression)

- Accuracy Obtained:

68.09%

- Confusion Matrix Screenshot:

```
Fold 1/5 — balanced_accuracy = 0.6814
Fold 2/5 — balanced_accuracy = 0.6806
Fold 3/5 — balanced_accuracy = 0.6817
Fold 4/5 — balanced_accuracy = 0.6813
Fold 5/5 — balanced_accuracy = 0.6794
— Ridge Classifier CV Summary —
  Mean Balanced Accuracy : 0.6809
  Total time              : 19.3s

Confusion Matrix (last fold)
Classes: ['High', 'Low', 'Medium']
[[ 3951   122   129]
 [ 4709 64795  4479]
 [17825 19375 10615]]
```

- What Went Wrong?:

Linear models try to draw simple straight lines to separate categories.

Farming data is too complicated for straight lines. It got super confused and misclassified over 17,000 Medium cases as High.

c. Failed Attempt 3

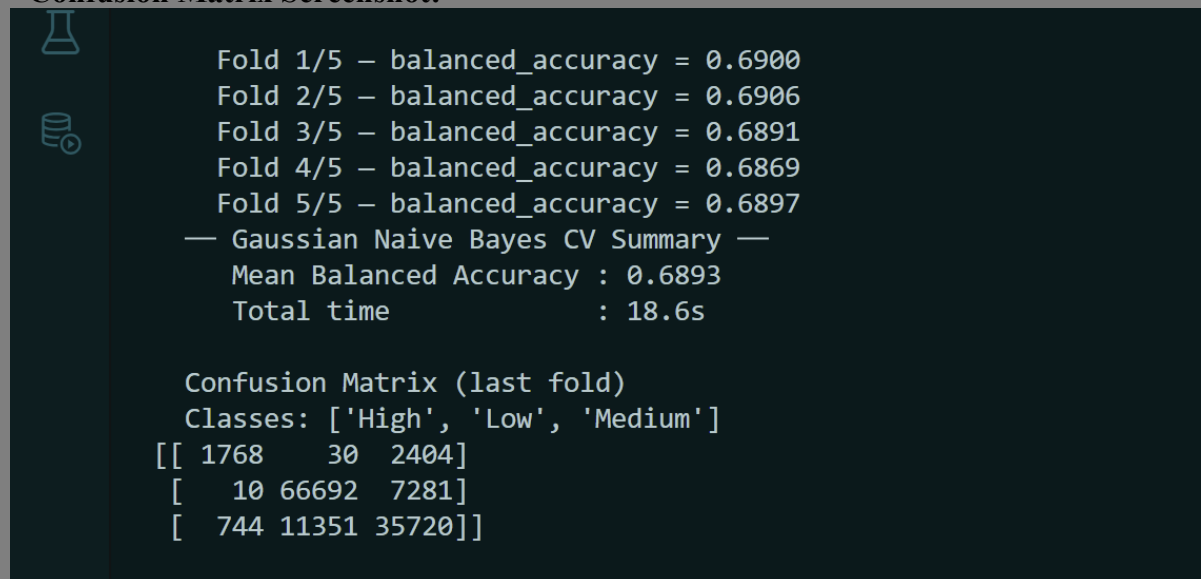
- Model Used:

Naive Bayes

- Accuracy Obtained:

68.93%

- Confusion Matrix Screenshot:



```

Fold 1/5 - balanced_accuracy = 0.6900
Fold 2/5 - balanced_accuracy = 0.6906
Fold 3/5 - balanced_accuracy = 0.6891
Fold 4/5 - balanced_accuracy = 0.6869
Fold 5/5 - balanced_accuracy = 0.6897
— Gaussian Naive Bayes CV Summary —
Mean Balanced Accuracy : 0.6893
Total time              : 18.6s

Confusion Matrix (last fold)
Classes: ['High', 'Low', 'Medium']
[[ 1768    30  2404]
 [   10 66692  7281]
 [   744 11351 35720]]
  
```

- What Went Wrong?:

Naive Bayes acts like every feature is completely independent, this assumption is what makes the algorithm naive. But in real farming, things like rainfall and soil moisture are closely connected and not independent. Because it couldn't understand those connections, it made a lot of mistakes, especially guessing Low when the answer was actually Medium.

d. Failed Attempt 4

- Model Used:

Decision Tree (Before Tuning)

- Accuracy Obtained:

94.63% (Overfitted) and 95.74% (Imbalanced)

- Confusion Matrix Screenshot:

5.4.1 Overfitted Matrix:



```
[ 0 47815  0]]
```



PHASE 2 – Decision Tree Tuning

Fold 1/5 – balanced_accuracy = 0.9456

Fold 2/5 – balanced_accuracy = 0.9481

Fold 3/5 – balanced_accuracy = 0.9475

Fold 4/5 – balanced_accuracy = 0.9447

Fold 5/5 – balanced_accuracy = 0.9453

— Overfitted Decision Tree CV Summary —

Mean Balanced Accuracy : 0.9463

Total time : 141.0s

Confusion Matrix (last fold)

Classes: ['High', 'Low', 'Medium']

```
[[ 3761    1  440]
```

```
[    0 72449 1534]
```

```
[ 506 1325 45984]]
```

5.4.2 Imbalanced Matrix:

```

Fold 1/5 – balanced_accuracy = 0.9557
Fold 2/5 – balanced_accuracy = 0.9586
Fold 3/5 – balanced_accuracy = 0.9591
Fold 4/5 – balanced_accuracy = 0.9562
Fold 5/5 – balanced_accuracy = 0.9575
— Imbalanced Decision Tree CV Summary —
Mean Balanced Accuracy : 0.9574
Total time              : 123.2s

Confusion Matrix (last fold)
Classes: ['High', 'Low', 'Medium']
[[ 3819      0   383]
 [      0 73477   506]
 [   298  1116 46401]]

```

- What Went Wrong?:

First, I let the tree grow as deep as it wanted (`max_depth=None`).
 It got 94.63%, but it memorized the training data and made weird mistakes.
 Then, I limited the depth to 15,
 but didn't tell it to care about the rare High class.

- What Was Improved?:

I put a strict limit on the tree's depth (`max_depth=20`)
 and used a setting called `class_weight='balanced'`
 to force the model to pay extra attention to the rare High class.
 This gave me my final, optimized score of 95.98%.

6. Final Model Selection

- Best Model:

Decision Tree Classifier

- Hyperparameters:

```
max_depth = 20,  
class_weight = 'balanced'  
min_samples_leaf = 10
```

- Why Selected? :

It was the only model that could understand the complex patterns in the data and handle the fact that the High category was extremely rare.

7. Leaderboard Performance

- **Kaggle Score:**

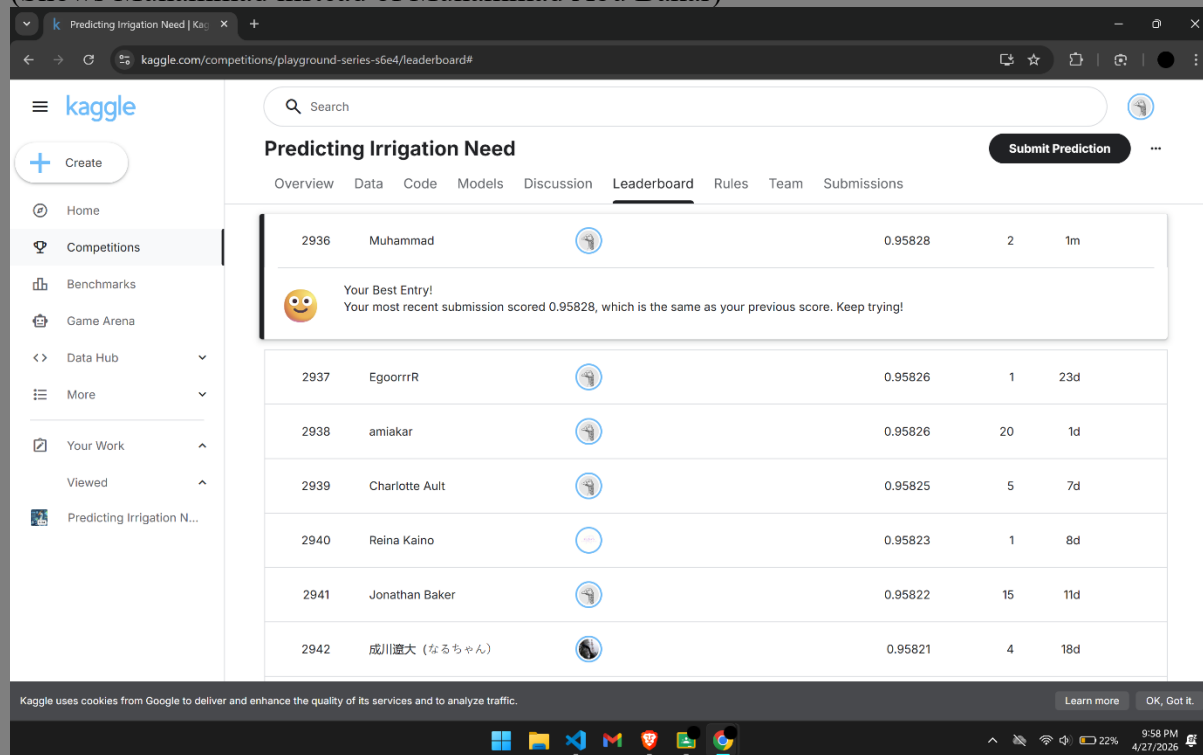
0.95828

- **Rank:**

2936

- **Screenshot Attached:**

(Shows Muhammad instead of Muhammad Abu Bakar)



The screenshot displays the Kaggle leaderboard for the 'Predicting Irrigation Need' competition. The page includes a sidebar with navigation links, a search bar, and a 'Submit Prediction' button. The leaderboard table lists participants with their IDs, names, scores, ranks, and submission times. A message box highlights the user's best entry.

Rank	Participant	Score	Rank	Time
2936	Muhammad	0.95828	2	1m
2937	EgoorrrR	0.95826	1	23d
2938	amiakar	0.95826	20	1d
2939	Charlotte Ault	0.95825	5	7d
2940	Reina Kaino	0.95823	1	8d
2941	Jonathan Baker	0.95822	15	11d
2942	成川遼大 (なるちゃん)	0.95821	4	18d

Message: Your Best Entry! Your most recent submission scored 0.95828, which is the same as your previous score. Keep trying!

8. Conclusion and Learnings

- Key Insights:

I learned that just throwing an AI algorithm at data doesn't work if you don't look at the data first. Because the High class was so rare, all the models failed until I forced them to balance the classes.

- Challenges Faced:

The dataset was huge 630,000 rows. Testing took a long time, and I had to be careful with my memory RAM, so my laptop wouldn't crash.

-Future Improvement:

If I had a faster computer, I would try a Random Forest model. It builds many Decision Trees and combines them together, which usually gives better accuracy and stops the model from memorizing data.